5-Day Advanced Interview Preparation Roadmap

This roadmap is intentionally designed to make the candidate think deeply about real-world engineering challenges instead of only completing beginner-level tutorials. The assignments simulate production-style scenarios involving frontend dashboards, real-time communication, backend APIs, device monitoring systems, schedulers, database integration, debugging, and business-oriented workflows. The candidate should implement all assignments independently and be able to explain architecture decisions, debugging approaches, database design, scalability concerns, and production troubleshooting during interviews.

DAY 1 — Advanced Frontend Foundations + REST Communication

Concepts To Learn In Detail

- Deep understanding of browser rendering pipeline, DOM tree creation, repaint vs reflow, and performance optimization.
- JavaScript execution context, call stack, event loop, microtask queue, callback queue, promises, closures, and asynchronous execution internals.
- Advanced Fetch API handling including timeout handling, retries, cancellation using AbortController, and API response transformation.
- HTTP protocol fundamentals including headers, caching, cookies, CORS, authentication tokens, and HTTP status code handling.
- Advanced CSS architecture including reusable layouts, dashboard responsiveness, dynamic theming, accessibility, and animation optimization.

Real-Time Assignment

Build a production-style Smart Traffic Operations Dashboard. The dashboard should dynamically display traffic incidents, signal statuses, congestion reports, and camera health status. Candidate must: - Create reusable UI sections - Implement API polling - Handle loading/error states - Add severity-based coloring - Implement sorting/filtering - Build responsive layouts for desktop and tablet - Create reusable JavaScript utility modules - Handle API failures gracefully

Backend System Design Question

Design a backend API structure for a traffic incident management system supporting filtering, pagination, and high concurrent traffic updates.

Frontend System Design Question

Design a scalable frontend dashboard architecture for displaying 10,000+ live traffic events without freezing the browser.

Interview Discussion Areas

- Explain API request lifecycle.
- How will you debug production API failures?
- What happens if database becomes slow?
- How would you scale the system?
- How will you handle real-time synchronization problems?
- What logging strategy would you use?
- How will you reduce frontend lag with huge data?

LeetCode Practice

Problem	Link
Two Sum	https://leetcode.com/problems/two-sum/
Valid Parentheses	https://leetcode.com/problems/valid-parentheses/

DAY 2 — Java Backend Engineering + Database Design

Concepts To Learn In Detail

- Spring Boot internals including DispatcherServlet flow, dependency injection lifecycle, bean scopes, and request handling.
- Advanced Java collections internals including HashMap bucket storage, collision handling, resizing, and concurrent collection behavior.
- Database schema design including normalization, indexing strategies, composite indexes, query execution plans, and transaction management.
- REST API versioning, DTO mapping, layered architecture, centralized exception handling, and validation patterns.
- Thread handling, synchronization basics, race conditions, and backend performance bottlenecks.

Real-Time Assignment

Build a complete Incident Management Backend System using Spring Boot. Candidate must: - Create layered architecture - Implement CRUD APIs - Add pagination and filtering - Add validation annotations - Implement centralized exception handling - Add database indexing - Create DTO mapping layer - Write production-style response objects - Implement search APIs for incidents/devices/signals - Create API documentation using Swagger

Backend System Design Question

Design a scalable backend architecture for storing and processing live traffic incidents coming from multiple cities simultaneously.

Frontend System Design Question

Design a frontend state management approach for handling multiple API modules like incidents, devices, signals, and reports.

Interview Discussion Areas

- Explain API request lifecycle.
- How will you debug production API failures?
- What happens if database becomes slow?
- How would you scale the system?
- How will you handle real-time synchronization problems?
- What logging strategy would you use?
- How will you reduce frontend lag with huge data?

LeetCode Practice

Problem	Link
Merge Two Sorted Lists	https://leetcode.com/problems/merge-two-sorted-lists/
Best Time to Buy and Sell Stock	https://leetcode.com/problems/best-time-to-buy-and-sell-stock/

DAY 3 — Real-Time Systems + Angular + WebSocket Engineering

Concepts To Learn In Detail

- WebSocket protocol lifecycle including handshake process, persistent connections, heartbeat handling, and reconnection strategies.
- Angular architecture deeply including modules, dependency injection, observables, RxJS streams, lazy loading, guards, and interceptors.
- TypeScript advanced typing system including interfaces, generics, decorators, and async programming.
- Frontend performance optimization strategies including virtual scrolling, change detection optimization, and lazy rendering.
- Real-time event streaming architecture and frontend synchronization challenges.

Real-Time Assignment

Build a Live Traffic Signal Monitoring Platform. Candidate must: - Use Angular with modular structure - Build reusable components - Integrate WebSocket communication - Handle reconnection automatically - Display real-time signal/device updates - Create operator notification panel - Build signal history timeline - Implement dynamic filtering/search - Use Angular Material professionally

Backend System Design Question

Design a WebSocket backend capable of pushing live updates to thousands of connected traffic operators.

Frontend System Design Question

Design a frontend architecture that can efficiently handle real-time updates from multiple WebSocket streams simultaneously.

Interview Discussion Areas

- Explain API request lifecycle.
- How will you debug production API failures?
- What happens if database becomes slow?
- How would you scale the system?
- How will you handle real-time synchronization problems?
- What logging strategy would you use?
- How will you reduce frontend lag with huge data?

LeetCode Practice

Problem	Link
Binary Search	https://leetcode.com/problems/binary-search/
Valid Anagram	https://leetcode.com/problems/valid-anagram/

DAY 4 — Node.js + MongoDB + Device Monitoring

Concepts To Learn In Detail

- Node.js event loop internals, asynchronous execution model, worker threads, and non-blocking I/O.
- NestJS/Express architecture including middleware, guards, interceptors, validation pipes, and modular service structure.
- MongoDB document modeling, aggregation pipelines, indexing strategies, replication basics, and query optimization.
- Scheduler systems using Quartz/Cron including retry logic, health checks, monitoring, and distributed scheduling concepts.
- Monitoring systems architecture including heartbeat tracking, alert generation, and failure detection.

Real-Time Assignment

Build a Device Health Monitoring System. Candidate must: - Create device heartbeat APIs - Store logs in MongoDB - Create scheduler-based offline detection - Generate automatic alerts - Build analytics APIs - Track last heartbeat timestamps - Implement retry handling - Build monitoring dashboards - Simulate multiple traffic devices

Backend System Design Question

Design a monitoring backend capable of tracking health status for 100,000 traffic devices in real time.

Frontend System Design Question

Design a frontend dashboard for monitoring thousands of online/offline devices while maintaining UI responsiveness.

Interview Discussion Areas

- Explain API request lifecycle.
- How will you debug production API failures?
- What happens if database becomes slow?
- How would you scale the system?
- How will you handle real-time synchronization problems?
- What logging strategy would you use?
- How will you reduce frontend lag with huge data?

LeetCode Practice

Problem	Link
Contains Duplicate	https://leetcode.com/problems/contains-duplicate/
Reverse Linked List	https://leetcode.com/problems/reverse-linked-list/

DAY 5 — System Design + Production Support + Interview Mastery

Concepts To Learn In Detail

- Microservices fundamentals, API gateway concepts, service communication patterns, and scalability considerations.
- Caching strategies including Redis basics, cache invalidation, and reducing database load.
- CI/CD pipeline fundamentals including Maven lifecycle, build pipelines, deployment automation, and rollback handling.
- Production debugging strategies including log tracing, monitoring dashboards, alert analysis, and incident troubleshooting.
- Requirement gathering, communication with business teams, production ownership mindset, and on-call responsibilities.

Real-Time Assignment

Build the Final Smart Traffic Monitoring Ecosystem by integrating: - Frontend dashboard - REST APIs - WebSocket communication - Database integration - Device monitoring - Scheduler jobs - Alert management - Incident management - Report generation Candidate must explain: - Complete architecture - API flows - Database relationships - Failure handling - Scalability improvements - Production deployment considerations

Backend System Design Question

Design the complete backend architecture for a state-wide traffic monitoring system supporting millions of events daily.

Frontend System Design Question

Design a highly scalable frontend architecture capable of handling multiple dashboards, reports, maps, and real-time feeds.

Interview Discussion Areas

- Explain API request lifecycle.
- How will you debug production API failures?
- What happens if database becomes slow?
- How would you scale the system?
- How will you handle real-time synchronization problems?
- What logging strategy would you use?
- How will you reduce frontend lag with huge data?

LeetCode Practice

Problem	Link
Flood Fill	https://leetcode.com/problems/flood-fill/
Number of Islands	https://leetcode.com/problems/number-of-islands/

Final Interview Preparation Advice

The candidate should focus on understanding how systems behave in production instead of only memorizing syntax. Interviewers for such roles usually evaluate:

- Ownership mindset
- Debugging capability
- API understanding
- Real-time system thinking
- Communication with business teams
- Ability to troubleshoot issues independently
- Understanding of scalable architecture
- Practical engineering thinking

The candidate should be able to explain: - End-to-end API flow - Frontend-backend communication - Database schema decisions - Real-time communication flow - Failure handling - Monitoring strategies - Scheduler responsibilities - Production troubleshooting approaches